iMovies

Full Stack Development with MERN
Project Documentation

Project Title	iMovies — Movie Ticket Booking System
Team Members	To be filled by the project owner
Institution / Department	To be filled
Project Dates	To be filled

Documentation note

This report is based on the supplied iMovies source snapshot. Team details, dates, screenshots, live URLs and observed test results must be completed by the project owner. Payments are simulated; no real payment processing is claimed.

1. Introduction

Project Title

iMovies — Movie Ticket Booking System

Introduction

iMovies is a full-stack web application for discovering movies, viewing showtimes, selecting seats and managing movie-ticket bookings. It demonstrates frontend/backend integration, database modelling, authentication, role-based administration and a simulated payment workflow.

2. Overview Project

Purpose

Provide a single web interface for movie discovery, showtime browsing, seat selection, demo checkout and booking management.

Features

Registration/login/logout; movie listing, filters and details; showtimes; seat layout and temporary holds; simulated payment success/failure; booking list/detail/cancellation; admin management of movies, theatres, screens, shows, bookings and users.

3. Architecture

Frontend

React 18, Vite, React Router, Axios and Bootstrap.

Backend

Node.js 18+ and Express 4 REST API.

Database

MongoDB accessed through Mongoose.

Authentication and security

JWT in an HTTP-only cookie, password hashing, request validation, Helmet, rate limiting and admin role checks.

4. Setup Instructions

Prerequisites

Node.js 18 or later, npm, Git and a reachable MongoDB database (for example, MongoDB Atlas).

Install

Clone the repository, run npm install in client and server folders, then configure server/.env from server/.env.example.

Environment variables

MONGODB_URI, JWT_SECRET and CLIENT_URL are required according to the README. VITE_API_URL may be used by the frontend. Verify exact variable names in the current example file. Never commit .env or real credentials.

Commands

Backend: cd server, npm install, npm run dev. Frontend in a second terminal: cd client, npm install, npm run dev.

5. Folder Structure

Main folders

client/src contains API wrappers, components, context, hooks, layouts and pages. server/src contains config, controllers, middleware, models, routes, services, utilities and validators. docs and Phase wise Doc contain project documentation.

6. Running the Application

Local URLs

Backend normally runs at http://localhost:5000 and Vite at http://localhost:5173. Confirm local configuration if ports differ.

Health check

Open /api/v1/health on the backend host.

7. API Documentation

Base prefix

/api/v1

Public and user APIs

GET /health; POST /auth/register; POST /auth/login; POST /auth/logout; GET /auth/me; GET /movies; GET /movies/:id; GET /movies/:id/shows; GET /theatres; GET /shows/:id; GET /shows/:id/seats; GET /bookings; POST /bookings/hold; GET /bookings/:id; POST /bookings/:id/cancel; POST /payments/demo/:bookingId.

Admin APIs

GET /admin/stats; GET /admin/users; GET /admin/bookings; POST /admin/bookings/:id/cancel; POST/PUT/DELETE /admin/movies; GET/POST/PUT/DELETE /admin/theatres; POST/DELETE /admin/screens; GET/POST/PUT /admin/shows.

Note

This is a concise endpoint overview. Query parameters, request bodies, response shapes and status codes should be confirmed from current route, validator and controller files.

8. Authentication

How it works

Users register and log in through the auth API. Passwords are hashed, and JWT authentication is stored in an HTTP-only cookie. Protected routes require authentication; admin operations require an admin role. Logout clears the authentication cookie.

9. User Interface

Screenshots to add

Home/movie listing; movie details and showtimes; seat selection; demo payment result; My Bookings; admin dashboard and management pages. Insert genuine screenshots from the running application and hide private data.

10. Testing

Test approach

Use the functional test cases in Phase wise Doc/Performance Testing/02-Functional-Test-Cases.md. Test registration, login/logout, movie browsing, showtimes, seat holds, payment success/failure, booking list/cancellation and admin authorization.

Results

No test is marked passed in this report solely from source inspection. Record actual results after running tests in the intended environment.

11. Screenshots or Demo

Live demo

To be filled and verified by the project owner.

GitHub repository

https://github.com/rnidhi4/iMovies — verify this is the correct public repository before submission.

12. Known Issues

Limitations

Payments and refunds are simulated; no email/SMS notifications or QR-code tickets; no show-delete route was found in the inspected route files; customer show listings are described as limited to a future seven-day window; performance measurements remain pending.

13. Future Enhancements

Possible improvements

Integrate a real payment gateway in sandbox mode; add email confirmations and QR tickets; add downloadable invoices; add reviews and watchlists; expand automated and concurrency tests; measure and optimise API/page performance; improve accessibility and mobile responsiveness.

Submission checklist

- Fill in team member names, institution, and project dates.
- Verify the public GitHub repository and live demo URLs.
- Add genuine application screenshots.
- Run functional tests and enter observed results.
- Measure performance before reporting any numerical claims.
- Review the report against the deployed version before submission.